

Finding the Right Layer

An AI-assisted study of peephole optimization in the Solidity compiler, and what happened when the patch met its reviewers

Philippe Laporte

Blockchain Technologies 6115
Information Systems Engineering
Concordia University

December 2, 2025

In memory of Laurie Hendren

Revised August 2026, after the pull request was reviewed, reworked and approved.

All measurements re-derived against `solc` commit `7eddaafa`.

ABSTRACT

This paper reports on an attempt to add new peephole optimization patterns to the Solidity compiler using a large language model as a research instrument, and on the reception those patterns received from the compiler's maintainers. Three candidate patterns were generated; one was unsound and died in the semantic test suite, one was a bad bargain that traded roughly 5,000 gas of deployment cost for 6 gas on a cold error path, and one was correct: the dispatcher guard that checks whether a call carries at least four bytes of calldata can be expressed as `gt(calldatasize(), 3)` rather than `iszero(lt(calldatasize(), 4))`, removing one instruction from essentially every contract the compiler emits.

The surviving pattern was submitted as a peephole rule and rejected in that form. A maintainer observed that the same effect was available one layer up, in Yul code generation, as a single line. That observation is the paper's central result. The agent was good at recognizing a local rewrite and poor at judging which layer of a multi-stage compiler should own it — and layer selection, not pattern discovery, was where the engineering value lay. The revised patch is one line, is approved, and is awaiting a runtime gas benchmark.

Along the way the paper corrects its own earlier analysis in two places. Measured against `evmone` across a 22-contract corpus, the optimization saves exactly one byte and exactly 3 gas per external call in every one of the 19 compilation units that emit a dispatcher, and nothing at all in the four that do not — not the 2 gas reported in the original submission, whose discrepancy was a measurement artifact rather than a property of real dispatchers. And the termination argument offered for the peephole fixed-point driver was wrong: the compiler contains a rule that *increases* code size, the driver's acceptance predicate admits no well-founded lexicographic order, and the 2017 maintainer worry about oscillation that the original paper dismissed as "vague and not rigorous" turns out to be well founded.

1 Introduction

Once upon a time I took a compiler design course with a Java Virtual Machine theme.^[1] We wrote code generation and early optimization for JOOS, a subset of Java, emitting class files through the Jasmin assembler.^[2] This was before ASM^[3] made bytecode manipulation routine. Our optimizer included peephole patterns^[4] driven by a fixed-point engine, and the course handed us a testing methodology I have never had cause to improve on:

1. argue carefully that each pattern is sound;
2. demonstrate that each pattern is realized correctly;
3. show statistically that the optimizer improves the generated code.

The concept carries to any architecture. A fixed-point peephole optimizer slides a window of stride one along generated code, tries a list of transformations that each recognize a specific instruction pattern with a known simplification, and stops when no transformation fires. The order of application matters — the rewrites do not commute — which is why iteration is needed before the process stabilizes. My old notes also promise that although an individual pattern need not shrink the code, the process terminates. Section 7 of this paper is, in a sense, an eighteen-year-late audit of that promise against a production compiler, and the promise does not survive contact intact.

This paper applies that three-phase methodology to the Solidity compiler, with one modern addition: a large language model used as a research instrument for code exploration and pattern discovery. The contributions are:

- A corrected account of where optimization actually happens in `solc`, including the cost models the compiler already contains (Section 2).
- A protocol for using an LLM agent productively on a large unfamiliar codebase, including the failure modes I hit (Section 3).
- One accepted optimization, with a soundness argument and measurements over a 22-contract corpus: 3 gas per external call and one byte of deployed code, uniformly, on every contract that dispatches on a selector (Section 4).
- One instructive rejection: a pattern whose cost reasoning was inverted, and what that reveals about agent judgment (Section 5).
- The layer-selection finding: the same rewrite expressed as a 60-line pattern matcher and as one line of code generation, and why the second is correct (Section 6).
- A refutation of my own earlier termination proof, and a repaired argument (Section 7 and Appendix A).

Solidity^[5] is the standard language for Ethereum smart contracts; `solc`^[6] is its reference compiler, targeting the Ethereum Virtual Machine^[7] — a stack machine with 256-bit words and big-endian representation. Both are open source, which is what makes a study like this possible: the patch discussed here is public, as is every review comment that shaped it.

2 Background: what optimizes what

2.1 Three layers, not two

My original submission described the optimizer as having "two layers, Yul and Peephole." That is wrong in both the count and the naming, and the error propagated into the rest of the argument, so it is worth correcting precisely. The documentation^[14] describes three kinds of optimization:

- **Codegen-based** — decisions made while lowering Solidity. In the legacy pipeline these are largely implicit and not configurable; in the IR pipeline codegen is deliberately thin, because nearly everything is deferred to the Yul optimizer.
- **Yul-based** — the powerful layer. It works across function calls, can compute side effects, and can reorder or delete calls. It runs over the whole contract only under `--via-ir`; in the legacy pipeline it applies just to internally generated Yul, such as the ABI coder.
- **Opcode-based** — operating on EVM assembly after code generation.

The second correction is that the peephole optimizer is not a layer. It is one component of the opcode-based module, alongside the common subexpression eliminator, the constant optimizer, the block deduplicator, the jumpdest remover, the inliner and the unused-pruner. `Assembly::optimiseInternal` runs all of them in a loop. Reading the peephole pass as though it were half the optimizer, as I originally did, badly overstates its importance and — more damagingly — hides the fact that its siblings are considerably more sophisticated than it is.

2.2 The peephole pass and its acceptance rule

The pass^[15] is a single sweep applying the first matching rule at each position. As of commit `7eddaafa` there are twenty entries, nineteen substantive and one `Identity` fallthrough that copies an unmatched item:

PushPop	OpPop	OpStop	OpReturnRevert
DoublePush	DoubleSwap	CommutativeSwap	SwapComparison
DupSwap	IsZeroIsZeroJumpI	EqIsZeroJumpI	DoubleJump
JumpToNext	UnreachableCode	TagConjunctions	TruthyAnd
DeduplicateNextTagSize3/2/1		Identity	

For comparison, LLVM's instruction-combining pass carries transformations on the order of a thousand. The gap is not a scandal — the two compilers work at very different levels and under very different risk tolerances — but it does indicate room.

What matters most for this paper is how the pass decides whether a sweep was worthwhile. After sweeping, it compares the new item sequence to the old:

```
if (m_optimisedItems.size() < m_items.size() || (
    m_optimisedItems.size() == m_items.size() && (
        bytesRequired(m_optimisedItems, ...) < bytesRequired(m_items, ...) ||
        numberOfPops(m_optimisedItems) > numberOfPops(m_items)
    )))
{
    m_items = std::move(m_optimisedItems);
    return true;
}
return false;
```

In words: accept if there are fewer items; or, at equal item count, if the encoding is shorter *or* there are more `POP`s. The `POP` count is a proxy for "we discarded a useless result." This disjunction is the acceptance predicate, and Section 7 shows it is the source of a real problem.

2.3 The compiler already reasons about gas

My original submission asserted that "there is no gas-cost-driven optimizer anywhere in the codebase," and built a recommendation on top of that claim. The claim is false, and the recommendation is better without it.

CORRECTION TO THE ORIGINAL SUBMISSION

The original paper supported this section with four quotations attributed to the Solidity documentation, among them "The optimizer mainly aims to reduce the bytecode size" and "Gas savings are not explicitly optimized for." None of those sentences appear anywhere in the documentation. They were produced by an LLM in the shape of quotations and I did not check them. They are withdrawn. What follows is quoted from the repository at commit `7eddaafa` and can be checked against it.

The documentation describes `--optimize-runs` as

a trade-off parameter between code size (deploy cost) and code execution cost (cost after deployment). A "runs" parameter of "1" will produce short but expensive code. In contrast, a larger "runs" parameter will produce longer but more gas efficient code.^[14]

That trade-off is implemented, not aspirational. `ConstantOptimizer` minimizes an explicit gas objective:

```
// libevmasm/ConstantOptimiser.h
return m_params.runs * _runGas
    + m_params.multiplicity * _repeatedDataGas
    + _uniqueDataGas;
```

where `runs` is documented in the same header as the "estimated number of calls per opcode over the lifetime of the contract." The `Inliner` takes the same `expectedExecutionsPerDeployment` parameter and uses it to decide whether inlining pays. And `libevmasm/GasMeter` exists precisely to estimate the gas cost of instruction sequences.

REFRAMING

The correct statement is not that `solc` ignores gas. It is that **the peephole pass is the odd one out**. Two of its immediate siblings in the same module optimize against a gas objective parameterized by `--optimize-runs`, while the peephole pass uses a size-and-POP proxy that cannot express "same size, cheaper." Any argument for making the peephole pass gas-aware does not need to introduce a new concept into the compiler; it needs to extend an existing one to a component that was left behind.

This matters for a concrete pattern I explored and abandoned. The rewrite `mul(x, 2k) → shl(k, x)` saves 2 gas (`MUL` costs 5, `SHL` costs 3) but for a small constant such as 32 it is size-neutral: `PUSH1 0x20; MUL` and `PUSH1 0x05; SHL` are both three bytes. The acceptance predicate therefore rejects it — same item count, same size, no new POPs — even though it is strictly cheaper to run. Only for constants needing a wider push (`PUSH2 0x0400; MUL` at four bytes versus `PUSH1 0x0a; SHL` at three) does the rewrite survive, and then it survives for the wrong reason. This is the acceptance predicate rejecting a real gas win because it cannot see gas, in a module whose other components can.

2.4 Where the bytes and the gas actually go

Two cost models matter, and the original submission conflated them.

Runtime code is what executes on every call. Its size is charged once, at deployment, at 200 gas per byte of deposited code; its instruction count is charged on every single call thereafter. **Creation code** — the constructor and the deployment wrapper — runs once and is transmitted as transaction calldata, so it is charged at calldata rates, not at the 200-gas deposit rate. My original text applied the 200 gas per byte figure to creation bytecode. It applies to the runtime code the constructor deposits.

The practical consequence for this paper is that removing one instruction from the dispatcher is charged twice in the right direction: once as roughly 200 gas of deployment, and again as a few gas on every call for the life of the contract. Contracts near the EIP-170 24 KB limit^[18] care about the first term independently.

3 Methodology

3.1 The agent as instrument, not as author

I began with no knowledge of Solidity or the EVM. I read the language specification^[8] in full, then the bytecode specification^[9], then the inline assembly^[10] and Yul^[11] documentation. I read the thousand lines of

PeepholeOptimiser start to finish. Everything else about the codebase I learned by asking an LLM targeted questions.

I claim there is no better way to approach a project of this size in a term, and I claim it with a caveat that I earned the hard way. The trick is not to let the model think for you but to let it answer questions you have already formed. It is a turbocharged literature search, not a collaborator.

Two failures are worth recording because both are characteristic.

Failure one: confident wrong answers to underspecified questions. Early on I asked whether the Solidity peephole optimizer had a fixed-point driver. The model said no. This misdirected weeks of my research toward building something that already existed. The question was bad: asked cold, it required the model to have indexed a specific control structure in a specific file. Once I supplied the file, the answer improved immediately — and the reason the first answer was wrong is itself instructive, because the driver is not in `PeepholeOptimiser` at all. It is in `Assembly::optimiseInternal`^[16], one level up.

Failure two: fabricated evidence. As documented in Section 2.3, the model produced four sentences formatted as quotations from the Solidity documentation, none of which exist. This is more dangerous than a wrong answer, because a fabricated quotation arrives wearing the costume of a checked fact. Every documentation quotation in this revision was verified by searching the repository at a named commit; the search commands are in [Appendix B](#).

3.2 Discovering candidate patterns

My first heuristic was to ask the model for peephole patterns in general. This failed completely: it produced a stream of plausible rewrites that, on inspection, never occur in compiler output, because the earlier optimizer stages have already eliminated them. Asking a model what *could* be optimized yields textbook patterns. Asking what is present in fully optimized output yields candidates.

The working protocol was therefore:

1. compile a contract at full optimization through both pipelines (`--optimize` and `--via-ir --optimize`);
2. dump the result with `--opcodes` ;
3. give the model only that output and ask what can be rewritten while preserving semantics;
4. verify every candidate actually occurs by recompiling and searching for it.

Step 4 is the one that does the work. The model also had to be repeatedly steered away from analyzing Yul when the question was about bytecode.

CORRECTION TO THE ORIGINAL SUBMISSION

The original text claimed the investigation "used hundreds of contract examples to deduce tons of useful patterns," and then two sentences later described starting with one contract. The second statement is the accurate one. The study examined a small number of contracts and produced three candidate patterns, of which one survived. The inflated claim was rhetorical and is withdrawn; Section 9 discusses what that small sample does and does not support.

3.3 Validation

Each candidate was carried through the same pipeline: implement the rule and its unit test, build, and run

```
./scripts/soltest.sh --no-semantic-tests --no-smt
```

then measure. Measurement compiles the contract, extracts the runtime bytecode, and executes it under `evmone`^[29] through the `evmc` driver:

```
solc --via-ir --optimize --bin-runtime T.sol
evmc run --vm libevmone.so --input <selector||args> @runtime.hex
```

Finally, because the change moves gas costs, the expectation suite must be refreshed with `isoltest --enforce-gas-cost --optimize --accept-updates`. A patch that produces no expectation churn here has probably not changed anything.

4 The function selector guard

4.1 The pattern

Every contract with at least one external function begins with a check that the incoming call carries enough calldata to contain a four-byte function selector. The compiler emitted this as a negated less-than. Written as EVM assembly, with the surrounding jump:

<i>; before</i>	<i>; after</i>
PUSH1 0x04 ; 3 gas	PUSH1 0x03 ; 3 gas
CALLDATASIZE ; 2 gas	CALLDATASIZE ; 2 gas
LT ; 3 gas	GT ; 3 gas
ISZERO ; 3 gas	
PUSH <tag>	PUSH <tag>
JUMPI	JUMPI
; 5 bytes, 11 gas	; 4 bytes, 8 gas

Figure 1. The guard before and after. Gas figures cover the condition only; the `PUSH <tag>` and `JUMPI` are common to both and are excluded.

In the compiler's own assembly listing the transformation reads as a single substitution inside the jump condition:

```
- jumpi(tag_1, iszero(lt(calldatasize, 0x04)))
+ jumpi(tag_1, gt(calldatasize, 0x03))
```

Note that the condition is consumed directly by `JUMPI`, which jumps *into* the dispatcher when the guard passes. There is no compiler-inserted inversion to interact with, which is what makes the rewrite as clean as it looks.

4.2 Soundness

Proposition 1. For all c in the range of `CALLDATASIZE`, `iszero(lt(c, 4))` and `gt(c, 3)` evaluate to the same value.

Proof. `CALLDATASIZE` pushes an unsigned 256-bit integer, so $c \in [0, 2^{256}-1]$. `LT` and `GT` are unsigned comparisons yielding 1 or 0, and `ISZERO` maps 0 to 1 and every nonzero value to 0, so on a Boolean input it is exactly negation. Then

$$\neg(c < 4) \equiv (c \geq 4) \equiv (c > 3)$$

where the second equivalence holds because c ranges over integers, so no value lies strictly between 3 and 4. Both the "4" and the "3" are compile-time literals, so neither wraps. The two forms leave the stack in the same state: each consumes nothing beyond the pushed literal and `CALLDATASIZE`, and each leaves exactly one Boolean word. ■

The integrality step is the only place the argument could fail, and it is the reason this rewrite does not generalize: `iszero(lt(x, k))` becomes `gt(x, k-1)` only for integer $k \geq 1$, and the `k = 0` case would underflow to $2^{256}-1$. Discharging the proposition mechanically over the full 256-bit domain takes four lines of SMT:

```
(declare-const cds (_ BitVec 256))
(assert (not (= (ite (bvult cds #x...04) #b0 #b1)
                (ite (bvugt cds #x...03) #b1 #b0))))
(check-sat) ; unsat - no counterexample exists
```

CORRECTION TO THE ORIGINAL SUBMISSION

The original paper discharged soundness by citing a ChatGPT share link as "a formal proof of soundness." A chat transcript is not a proof, is not archival, and is not checkable by a reviewer — the links in the original are already partly dead. The argument above is stated in the paper and the SMT formulation can be re-run by anyone. Original submission also stated the property over unbounded integers; the bit-vector formulation above is the one that actually models the EVM.

4.3 Measurements

The following were produced in August 2026 against `solc` commit `7eddaafa`, by building the compiler twice — once with the change and once with the single line reverted — and executing both outputs under `evmone` 0.18. The contract is the minimal trigger case:

```
pragma solidity ^0.8.20;
contract TriggerSelectorGuard {
    function foo(uint256 x) external pure returns (uint256) { return x + 1; }
}
```


MEASUREMENT	BEFORE	AFTER	Δ
Guard encoding (bytes)	5	4	-1
Code, excluding metadata (bytes)	96	95	-1
Deployed runtime code (bytes)	187	186	-1
Deployment, code deposit (gas)	37,400	37,200	-200
Call to <code>foo(1)</code> , accepted (gas)	180	177	-3
Call with 3 bytes, rejected (gas)	50	47	-3
Call with 0 bytes, rejected (gas)	50	47	-3

Table 1. Measured effect of the optimization. Gas figures are `evmone` executions of the runtime bytecode at the Cancun revision. Code deposit is the 200 gas/byte charge on runtime code size. The "before" byte counts have the four-byte metadata artifact of Section 4.4 removed; raw totals were $96 + 93 + 2 = 191$ for the patched build versus $95 + 89 + 2 = 186$ for the clean one.

Across a corpus

A single contract establishes the mechanism but not its reach, which is the third phase of the methodology in Section 1 and the phase the original submission skipped. I therefore compiled a corpus of 22 contracts — spanning minimal cases, an ERC-20, a proxy, libraries, inheritance, interfaces, overloads, custom errors, string and struct parameters, and deliberate controls — with both compilers, and executed every resulting runtime object under `evmone`. The corpus yields 23 compilation units, since one file also produces a library.

COMPILATION UNITS	COUNT	Δ BYTES	Δ GAS, REJECTED CALL	Δ GAS, DISPATCHED CALL
With a selector dispatcher	19	-1 (all)	-3 (18 of 19)	-3 (19 of 19)
Without one (controls)	4	0 (all)	0 (all)	n/a

Table 2. Corpus results. The effect is not merely typical but exactly uniform: every unit that emits a dispatcher saves precisely one byte and precisely three gas, and every unit that does not is untouched.

The four controls are the useful part of this table, because they confirm the optimization fires exactly where the model says it should and nowhere else. A library, a contract with only internal functions, a contract with only `receive()`, and a contract with only `fallback()` are all bit-for-bit identical under the two compilers. This follows from the template: the guard sits inside a `<?+cases>` conditional and is simply not emitted when a contract has no external function cases to dispatch to.

The single exception in the table is the proxy contract on the rejected-call path, which reports no difference. This is a limitation of the measurement harness rather than of the optimization. `evmc run` executes bytecode without host state, so the `delegatecall` in that contract's fallback is not modelled; both builds report an identical figure close to 63/64 of the gas limit, which is the mock being measured and not the guard. The same contract's dispatched path, which does not reach the fallback, shows the expected -3 gas, and its code is one byte shorter like every other dispatcher.

CORRECTION TO THE ORIGINAL SUBMISSION

The original paper reported "raw savings: 3 gas, measured via evmone: 2 gas," and explained the gap by asserting that "real-world dispatcher structure reduces the effective improvement by ~1 gas." That explanation was invented to account for a bad measurement. The saving is 3 gas, on every path, exactly as the instruction costs predict; the guard is straight-line code with no structure available to absorb a gas unit. When theory and measurement disagree by precisely the cost of the instruction you removed, the measurement is the thing to re-check.

4.4 A pitfall worth recording

The first size comparison I ran during this revision reported a 5-byte saving, which would have been a lovely result and was entirely spurious. Building the "before" compiler required editing a tracked source file, which made the working tree dirty, which made `solc` stamp its version string as `...+commit.7eddaafa.mod` instead of `...+commit.7eddaafa`. That version string is embedded in the metadata CBOR appended to every contract, so the "before" binary carried four extra bytes that had nothing to do with the dispatcher.

Any bytecode size comparison across two builds of a compiler must either strip the metadata or ensure both builds report identical version strings. The instruction-level diff, which is immune to this, showed exactly one instruction removed and every subsequent offset shifted by one — which is the honest 1-byte result.

5 A rejected pattern, and what it reveals

Of the three candidates the agent produced, one was eliminated by the semantic test suite as unsound — the intended outcome of phase 2 of the methodology, and unremarkable. The second is more interesting, because it was *sound* and still wrong.

The proposed `ErrorDispatcherStandardPanic` pattern canonicalized the construction of a panic selector, replacing a computed push with a single wide literal:

FORM	SEQUENCE	BYTES	RUNTIME GAS
Before	PUSH4, PUSH1, SHL, PUSH0, MSTORE	10	~11
After	PUSH32, PUSH0, MSTORE	35	~5
Δ		+25	-6

Table 3. The rejected panic-dispatcher rewrite. The 25 additional bytes cost roughly 5,000 gas at deployment, unconditionally; the 6 gas saving is realized only when a panic actually fires.

The agent presented this as a "classic space–time trade-off," and in the abstract it is one. But the trade is catastrophic in the direction it goes. Panic paths are, by construction, the paths a correct contract never takes. The rewrite therefore pays about 5,000 gas of deployment cost on every deployment, forever, to save 6 gas on executions that ideally never happen. A rational contract author would take that trade only if they expected to panic more than eight hundred times per deployment, which is another way of saying never.

FINDING

The agent's error was not arithmetic — its byte counts and gas figures were correct, and I checked them. The error was that it applied a cost-model template ("space for time is a classic trade") without weighting the two sides by their execution probability. Local correctness reasoning was reliable throughout this study; global cost reasoning was not. The distinction matters because the first is easy to verify automatically and the second is not.

It is worth noting that the compiler already has the machinery to express exactly the judgment the agent failed to make. As Section 2.3 established, `expectedExecutionsPerDeployment` is precisely a weight for "how often will this run relative to being deployed." The agent proposed a trade-off in a component that has no access to that parameter, having never asked whether the parameter existed.

6 The layer question

This section reports what happened after submission, and is the reason this paper was revised rather than left standing.

6.1 What was submitted, and what came back

The surviving pattern was implemented as a peephole rule named `FunctionSelectorGuard`: a matcher recognizing the five-item sequence `PUSH1 0x04, CALLDATASIZE, LT, ISZERO` preceded by a `Tag`, and a rewriter emitting the four-item replacement. The `Tag` restriction existed to ensure the sequence really was the dispatcher guard and not a coincidentally similar comparison arising from array bounds checking or ABI decoding. It was submitted as pull request #16315^[23] with a supporting issue^[24], following the project's review checklist^[25].

A maintainer replied within the week:

There's also likely a significantly easier approach to achieve the same effect by changing the Yul codegen directly.

They were right, and the correction is total. The guard is not an emergent property of code generation that has to be discovered by scanning output. It is emitted from one template, in one function, in one file. The whole of the reworked patch is:

```
// libsolidity/codegen/ir/IRGenerator.cpp, in dispatchRoutine()
- <?+cases>if iszero(lt(calldatasize(), 4))
+ <?+cases>if gt(calldatasize(), 3)
```

The peephole rule was deleted. The revised patch was approved in August 2026 and is, at the time of writing, awaiting results from a runtime gas benchmarking harness the team is building — a reviewer's request that is, given Section 4.3, exactly the right thing to ask for.

6.2 Why the higher layer is right

	AS A PEEPHOLE RULE	AS CODE GENERATION
Size of change	~60 lines plus tests	1 line plus tests
Precondition	Pattern must be recognized in output, with a <code>Tag</code> guard to avoid false positives	None; the code is simply not emitted
Ordering sensitivity	Subject to the acceptance predicate and to interaction with other rules	None
Coverage	Only sequences that survive earlier passes in the exact expected shape	Every dispatcher, unconditionally
Failure mode	Silently stops firing if the emitted shape changes	Compile error or a failing test
Where it is tested	Peephole unit tests	Existing dispatcher expectations across the whole suite

Table 4. The same semantic change at two layers.

The decisive column is the failure mode. A peephole rule that stops matching does not announce itself; it silently stops optimizing, and the only evidence is a gas expectation that nobody notices moved in the wrong direction. The codegen change cannot fail silently, because the string it edits is the thing that produces the code.

There is a general principle here, and it is not new — it is the compiler engineering instinct that a rewrite belongs at the highest layer that has enough information to make it. What is new, or at least newly urgent, is that this instinct is exactly what an LLM agent does not have. The agent was given bytecode and asked what could be improved in bytecode. It answered that question well. It had no way to ask the prior question: *should this be fixed in bytecode at all?*

CENTRAL FINDING

The agent's contribution was recognizing that `iszero(lt(c, 4))` and `gt(c, 3)` are interchangeable and that the former appears in every dispatcher. That is genuine and it is what got merged. The engineering contribution — deciding that this belongs in code generation and not in a pattern matcher — came from a human reviewer who knew the pipeline. The framing that fits my results is not "AI generates optimizer patterns" but **AI proposes local rewrites; layer selection remains human**, and layer selection is where the durability of the change is decided.

6.3 The prediction I did not notice I had made

The original submission, in its discussion of where gas-aware transforms should go, concluded that we should "move gas-aware transforms to the Yul optimizer instead," and argued this was "what the Solidity team would approve of most easily," listing the upsides as approval likelihood, safety, generality and better architecture. That paragraph was written about a different pattern — the `mul / shl` strength reduction — and I did not apply it to the pattern I was actually submitting. The reviewer applied it for me six days later. The analysis was right; I simply had it filed under future work instead of under the patch in front of me.

7 Termination of the fixed-point driver

The original submission argued that the peephole driver's iteration counter is unnecessary, and offered an agent-generated proof in an appendix to support removing it. That argument is wrong. This section replaces it. The correction is the most substantial change in this revision, and I think the resulting analysis is more interesting than the claim it replaces.

7.1 What the driver actually is

There are two nested loops, and the original paper conflated them:^[16]

```
for (unsigned count = 1; count > 0;)      // outer: whole opcode module
{
    count = 0;
    if (runInliner)      Inliner{...}.optimise();
    if (runJumpdestRemover) { if (jumpdestOpt.optimise(...)) count++; }

    if (runPeephole)
    {
        PeepholeOptimiser peepOpt{m_items, m_evmVersion};
        while (peepOpt.optimise())      // inner: peephole to fixed point
        {
            count++;
            assertThrow(count < 64000, OptimizerException,
                        "Peephole optimizer seems to be stuck.");
        }
    }
    if (runDeduplicate) { ... }
    if (runCSE)          { ... }
    ...
}
```

Three observations the original missed. First, `count` is *reset to zero* at the top of every outer iteration, so the 64000 bound limits peephole passes within a single outer round, not globally. Second, `count` is shared with the jumpdest remover, which increments it first, so the peephole budget is 64000 minus that. Third, and most importantly, the guard protects only the inner loop; nothing bounds the outer loop, in which the peephole pass, the CSE, the block deduplicator and the inliner can feed one another. If divergence is a worry anywhere, the outer loop is the more plausible place, and it has no guard at all.

The guard was added in 2017^[19]. The review exchange the original paper quoted, and dismissed, ran:

Q: Do you have a test case which triggered a[n infinite] loop?

A: No, but the two conditions (number of pops and bytecode size) could oscillate.

The original paper called this "vague and not rigorous." It is terse, but it is correct, and it identifies the exact mechanism. What follows vindicates it.

7.2 The refutation

The agent-generated proof rested on a single premise, stated as: "All patterns either reduce code size or leave it unchanged. No pattern increases code size." The premise is false, and the counterexample is in the rule list.

`OpPop` rewrites a side-effect-free operation whose result is immediately discarded into one `POP` per operand:

```
// libevmasm/PeepholeOptimiser.cpp
if (_pop == Instruction::POP && _op.type() == Operation)
{
    auto const& info = instructionInfo(_op.instruction(), EVMVersion());
    if (info.ret == 1 && !info.sideEffects)
    {
        for (int j = 0; j < info.args; j++)
            *_out = {Instruction::POP, _op.debugData()};
        return true;
    }
}
```

For a two-operand instruction this is size-neutral. But `ADDMOD` and `MULMOD` take three operands, return one value, and have no side effects — the instruction table records `ADDMOD` as `{args 3, ret 1, sideEffects false}`. So:

WINDOW	ITEMS	BYTES	POPS
<code>ADDMOD, POP</code>	2	2	1
<code>POP, POP, POP</code>	3	3	3

Table 5. `OpPop` applied to `ADDMOD` increases item count and byte count while increasing the `POP` count. The premise of the original termination argument fails here.

7.3 Why no simple measure works

Recall the acceptance predicate from Section 2.2. Writing n for item count, b for encoded bytes and p for `POP` count, a sweep is accepted iff

$$n' < n \vee (n' = n \wedge (b' < b \vee p' > p))$$

Consider the regime where n has stopped decreasing, which it must eventually, being a non-increasing natural number. In that regime each accepted sweep decreases b or increases p — but not necessarily both, and the predicate places no constraint at all on the other quantity. So:

- a sweep accepted for $p' > p$ may have $b' > b$;
- a sweep accepted for $b' < b$ may have $p' < p$.

Neither lexicographic order $(n, b, -p)$ nor $(n, -p, b)$ is therefore decreasing on every accepted step, and no weighting of b against p can fix this, since the predicate permits arbitrary movement in the unconstrained coordinate. The acceptance predicate, considered on its own, does not induce a well-founded order.

Worse, both directions of the oscillation are instantiated by rules that exist. `OpPop` on `ADDMOD` is a bytes-up, `POP`s-up transition. `PushPop`, which deletes a `PUSH/POP` pair outright, is a bytes-down, `POP`s-down transition. The 2017 reviewer's two oscillating quantities are not hypothetical; they are the two coordinates of the predicate, and the rule set can move in both directions along each.

7.4 What can actually be proved

Termination does hold, but for a reason the size argument never touches.

Proposition 2. The inner peephole loop terminates.

Proof sketch. Assign to each item sequence the multiset of its non-POP instructions, ordered by the multiset extension of any total order on opcodes. Every rule in the list either deletes non-POP instructions, or replaces them with strictly fewer non-POP instructions, or replaces them with POPs. Critically, no rule *introduces* a non-POP instruction that was not already in its input window: OpPop emits only POPs, DoublePush emits a DUP in place of a PUSH it consumed, and the remaining rules are deletions or reorderings. The multiset is therefore non-increasing on every sweep and strictly decreasing on any sweep that fires a non-reordering rule. The purely reordering rules (CommutativeSwap, SwapComparison) move toward a fixed canonical form and cannot fire twice on the same window. Hence no infinite sequence of accepted sweeps exists. ■

This is a sketch and not a machine-checked proof; in particular the claim that the canonicalizing rules cannot cycle deserves a proper argument per rule, and DeduplicateNextTag interacts with tag identity in ways I have not fully characterized. I state it at the level of rigor I can actually support, which is a change from the original.

7.5 Revised recommendation

RETRACTION

The original paper proposed "an eventual short and sweet PR to get rid of the count and let it loop forever with confidence that it will converge." I withdraw that proposal. The confidence was unfounded: it rested on a premise contradicted by OpPop, and the property that does hold depends on a per-rule argument about opcode creation that no future contributor adding a rule would think to check. The guard is cheap and it encodes exactly the invariant that is not otherwise enforced. It should stay.

Two things would be genuine improvements, and neither is the deletion I proposed. First, the guard's message — "Peephole optimizer seems to be stuck" — could state the invariant it is protecting, so that a contributor who trips it learns what they broke. Second, and more substantially, the acceptance predicate could be replaced by a single monotone cost function of the kind ConstantOptimiser already uses. That would make progress well-founded by construction, would let size-neutral but cheaper rewrites such as `mul → shl` be accepted on their merits, and would bring the peephole pass into line with its siblings. The 64000 guard is not the problem; it is a symptom of the proxy measure underneath it.

8 Related work

Schett and Nagele^[20] populate a smart contract compiler's peephole optimizer automatically, using SMT-based superoptimization to search for optimal instruction sequences over small windows and to prove the resulting rewrites equivalent.

CORRECTION TO THE ORIGINAL SUBMISSION

The original paper cited this work as confirming that "AI is best suited at finding patterns in general, and peephole patterns in particular." It confirms nothing of the sort. Schett and Nagele use constraint solving, not machine learning; there is no learned model anywhere in their approach. Citing an SMT paper as evidence for an LLM methodology inverted its actual contribution, which is that patterns found this way come with proofs attached — precisely the property my agent-generated patterns lacked, and the reason one of the three was unsound.

GASOL^[27], from the COSTA group, optimizes EVM bytecode with a gas objective. Its reported results are instructive: across thirty contracts, gas savings were 0.43% for the Yul optimizer alone, 0.84% for GASOL applied to unoptimized bytecode, and 1.28% for GASOL applied on top of the Yul optimizer. Two conclusions follow. The superoptimizer roughly doubles the Yul optimizer's savings on the same input, and the two compose rather than compete. The cost is time: the Yul optimizer runs in seconds, GASOL does not.

Why have these findings not landed in `solc`? Three reasons, and my experience with a single one-line patch corroborates all three at small scale.

Trust and proof. In smart contracts, a miscompilation is a fund loss. The project's review checklist^[25] demands that changes be correct, tested and rebased, and reviewers ask for demonstrated equivalence. Automatically generated rules arrive as a large set of individually non-obvious rewrites, and reviewing a thousand of them is not a thousand times easier than reviewing one.

Confluence. Schett and Nagele report that their generated rule sets are non-confluent: the final output depends on application order. That is the same class of problem as Section 7.3, scaled up. A compiler whose output depends on rule ordering is hard to reason about and hard to keep reproducible.

Cost. Superoptimization is a search. It cannot run on every basic block of every compilation. The intended division of labour — a slow offline discovery phase feeding a fast linear-time peephole pass — is sound, but it means the discovery phase's output must survive human review before it is worth anything, which returns us to the first reason.

On scope, a 2022 study^[26] found that roughly 23% of analyzed deployed contracts contain inline assembly. This bears directly on where optimizations should live: the opcode-based pass is the only stage that sees Yul-generated code, legacy-generated code and hand-written user assembly alike. A codegen-layer fix like the one in this paper cannot help hand-written assembly by construction. That is a real argument for the peephole layer — just not for this particular pattern, which no user writes by hand.

9 Threats to validity

Sample size and its limits. The corpus of Section 4.3 is 22 hand-written contracts, not a sample of deployed code. Because the effect turned out to be exactly uniform — one byte and three gas, every time a dispatcher is emitted — the risk this poses is small: the quantity being measured has no variance to sample. The corpus was chosen to falsify rather than to confirm, which is why it includes four contracts where the optimization must *not* fire, and it would have exposed a pattern that failed on inheritance, overloads, custom errors, structs or proxies. What it cannot support is a claim about aggregate savings across the deployed contract population; that would require weighting by real deployment and call frequencies, which I have not done. The honest statement is per-contract and per-call, and that is the statement the paper makes.

Single implementation. All gas figures come from `evmone`. Gas schedules are consensus rules, so agreement across clients is expected rather than assumed, but I did not verify it. The reviewer's request to wait for a proper benchmarking harness is the right correction to this weakness. Section 4.3 also documents one case — the proxy's fallback path — where the standalone runner's lack of host state makes a measurement meaningless; a harness with real state would not have that gap.

Agent-assisted analysis. Parts of this study were produced with LLM assistance, and Sections 2.3, 4.3, 5 and 7 each document a case where that assistance produced confident, well-formatted, wrong output that I initially

accepted. I have re-derived every factual claim in this revision against a named commit, but I cannot rule out that this process has its own residue.

Pipeline scope. The accepted change is in the IR pipeline's code generator. The legacy pipeline emits its dispatcher separately and is unaffected.

10 Conclusion

I set out to add peephole patterns to the Solidity compiler using an LLM agent as the pattern-discovery engine, expecting the contribution to be the patterns. Three candidates were produced. The semantic test suite killed one for unsoundness. Cost analysis killed a second, which was sound but paid roughly 5,000 gas of deployment to save 6 gas on a path that correct programs never take. The third was real, and it is now approved — as one line in the IR code generator, not as the peephole rule I wrote.

That relocation is the result. The agent identified a true and useful equivalence and placed it at the wrong layer, and no amount of additional pattern generation would have corrected that, because the layer question is not visible from the bytecode the agent was reading. A reviewer who knew the pipeline saw it in six days. The division of labour that emerged is a reasonable one to plan around: agents are effective at proposing local rewrites and at the mechanical work of implementing and testing them, and they are not yet reliable at deciding where a change belongs or at weighting costs by how often code actually runs.

The secondary finding is a caution about the same instrument. Every substantive error this revision corrects — fabricated documentation quotations, an unreproducible gas figure with an invented explanation, a termination proof resting on a premise the source code contradicts, and a citation that inverted the contribution of the work it cited — came from accepting fluent output without checking it. The methodology from a compilers course in the previous decade held up perfectly: argue soundness, demonstrate correct realization, measure. It was the steps where I skipped straight to the write-up that failed.

Future work. The strength-reduction family (`mul/div/mod` by powers of two) remains unimplemented and belongs in the Yul optimizer, where a cost model exists. Replacing the peephole acceptance predicate with a monotone gas-and-size cost function, as suggested in Section 7.5, would both simplify the termination story and unlock size-neutral gas wins. And the outer optimization loop deserves the termination analysis that this paper gave the inner one.

A Appendix: the original termination argument, annotated

For the record, this is the structure of the agent-generated argument that appeared as Appendix A of the original submission, with the defects marked. It is reproduced because the failure mode is more instructive than the conclusion.

The argument enumerated the peephole rules and annotated each with a claim of the form "reduces code size; cannot re-enable itself or others," then concluded in five steps:

1. *List of patterns.* The list included `IsZeroIsZeroRJumpI`, `EqIsZeroRJumpI`, `DoubleRJump` and `RJumpToNext`. These were the EOF relative-jump variants, removed from the compiler along with the rest of experimental EOF support in 0.8.36. It also listed `FunctionSelectorGuard` among the existing rules — my own proposed pattern, which had never been merged. **The inventory described a compiler that did not exist.**
2. *Pattern effects and monotonicity.* Every entry asserts non-increase in code size. **False for `OpPop`**, as Section 7.2 shows.
3. *Interaction and cycles.* "All patterns either reduce code size or leave it unchanged... since they do not increase code size, and all other patterns are strictly reducing, the process cannot cycle indefinitely." **This is the load-bearing step and it is invalid twice over:** the premise is false, and even were it true, a set of non-increasing transformations can cycle. Non-increase is not well-foundedness. The argument also never mentions the acceptance predicate, which is the only thing that actually gates progress.
4. *Bounded steps.* Concludes a bound proportional to initial code length. **Follows from step 3, hence unsupported.**
5. *Conclusion.* "The count guard is not strictly necessary for correctness, though it remains a good safety net for future pattern additions." **Retracted** — see Section 7.5. The concessive clause is, ironically, the correct part.

The argument reads as a proof. It has numbered sections, a case analysis, a stated conclusion and a summary. What it lacks is any step that could have failed — every claim about every rule was asserted rather than derived from the rule's source, and one check against `PeepholeOptimiser.cpp` would have falsified the premise. That is the signature to watch for: a proof whose case analysis never consults the artifact it is reasoning about.

B Appendix: reproduction

All figures in Section 4.3 were produced against `solc` at commit `7eddaafa` (August 2026) and `evmone` 0.18.

```

# build both compilers: with the change, and with the one line reverted
cd solidity/build && make -j solc

# emit runtime bytecode for the trigger contract
solc --via-ir --optimize --bin-runtime T.sol

# confirm the emitted guard
solc --via-ir --optimize --asm T.sol | grep calldatasize
#   -> jumpi(tag_1, gt(calldatasize, 0x03))

# measure: accepted call, foo(1)
evmc run --vm libevmone.so \
  --input 2fbabd38000...001 @runtime.hex
#   before: Gas used: 180      after: Gas used: 177

# measure: rejected call, 3 bytes of calldata
evmc run --vm libevmone.so --input 2fbabd @runtime.hex
#   before: Gas used: 50      after: Gas used: 47

# corpus sweep: both compilers over every contract, sizes and gas
for f in corpus/*.sol; do
  for c in solc-before solc-after; do
    ./$c --via-ir --optimize --no-cbor-metadata --bin-runtime "$f"
  done
  evmc run --vm libevmone.so @before.hex # rejected call
  evmc run --vm libevmone.so --input $SEL @after.hex # dispatched call
done

# verify documentation quotations rather than trusting them
grep -rn "trade-off parameter between code size" docs/

```

Caveat one. Reverting the source line dirties the working tree, which appends `.mod` to the compiler version string embedded in contract metadata and inflates the apparent size difference by four bytes. Pass `--no-cbor-metadata`, compare disassembly, or strip the trailing CBOR rather than comparing raw binary lengths. See Section 4.4.

Caveat two. Build both compilers once and keep the two binaries. Rebuilding between measurements invites exactly the inconsistency described above, and the corpus sweep needs each binary many times.

References

1. Java virtual machine. en.wikipedia.org/wiki/Java_virtual_machine
2. Jasmin — Java assembler. github.com/davidar/jasmin
3. ASM — Java bytecode manipulation framework. asm.ow2.io
4. Peephole optimization. en.wikipedia.org/wiki/Peephole_optimization
5. Ethereum. ethereum.org
6. The Solidity compiler. github.com/argotorg/solidity
7. Ethereum Virtual Machine. ethereum.org/developers/docs/evm/
8. Solidity documentation, v0.8.30. docs.soliditylang.org/en/v0.8.30/
9. Antonopoulos & Wood, *Mastering Ethereum*, ch. 13: The EVM. github.com/ethereumbook/ethereumbook/blob/develop/13evm.asciidoc
10. Solidity documentation: inline assembly. docs.soliditylang.org/en/v0.8.30/assembly.html
11. Solidity documentation: Yul. docs.soliditylang.org/en/v0.8.30/yul.html
12. Monad. monad.xyz
13. Go Ethereum's JIT EVM (2016). blog.ethereum.org/2016/06/02/go-ethereums-jit-evm — describing an experimental JIT that was subsequently removed from go-ethereum. Contrary to the original submission, go-ethereum is the dominant execution client, not a minor project, and it does not ship a JIT today.
14. Solidity documentation: internals of the optimizer. docs.soliditylang.org/en/v0.8.30/internals/optimizer.html
15. `libevmasm/PeepholeOptimiser.cpp`. github.com/argotorg/solidity/blob/develop/libevmasm/PeepholeOptimiser.cpp
16. `libevmasm/Assembly.cpp`, `optimiseInternal`. github.com/argotorg/solidity/blob/develop/libevmasm/Assembly.cpp
17. Ethereum documentation: gas and fees. ethereum.org/developers/docs/gas/
18. EIP-170: contract code size limit. eips.ethereum.org/EIPS/eip-170
19. solidity PR #3077, introducing the peephole iteration guard (2017). github.com/argotorg/solidity/pull/3077
20. M. Schett and J. Nagele. Populating the Peephole Optimizer of a Smart Contract Compiler. *FMBC 2020*, OASICS vol. 84. [drops.dags
tuhl.de/storage/01oasics/oasics-vol084-fmbc2020/OASICS.FMBC.2020.3/OASICS.FMBC.2020.3.pdf](https://drops.dagstuhl.de/storage/01oasics/oasics-vol084-fmbc2020/OASICS.FMBC.2020.3/OASICS.FMBC.2020.3.pdf)
21. EIP-150: gas cost changes for IO-heavy operations. eips.ethereum.org/EIPS/eip-150
22. EIP-145: bitwise shifting instructions (Constantinople), which introduced `SHL / SHR`. eips.ethereum.org/EIPS/eip-145 — cited in the original submission as "Berlin," which is a different hard fork.
23. solidity PR #16315, the patch discussed in this paper. github.com/argotorg/solidity/pull/16315
24. solidity issue #16316. github.com/argotorg/solidity/issues/16316
25. Solidity review checklist. github.com/argotorg/solidity/blob/develop/ReviewChecklist.md
26. S. Chaliasos et al. A Study of Inline Assembly in Solidity Smart Contracts. *OOPSLA 2022*. [chaliasos.com/assets/papers/inline-oopsla2
2.pdf](https://chaliasos.com/assets/papers/inline-oopsla22.pdf)
27. GASOL: gas analysis and optimization toolkit, COSTA group. github.com/costa-group/gasol-optimizer
28. EIP-3855: `PUSH0` instruction. eips.ethereum.org/EIPS/eip-3855
29. evmone — C++ EVM implementation. github.com/ethereum/evmone
30. `libevmasm/ConstantOptimiser.h` and `libevmasm/GasMeter.h`, the compiler's existing gas cost models. [github.com/argotor
g/solidity/tree/develop/libevmasm](https://github.com/argotorg/solidity/tree/develop/libevmasm)